How to Deploy in One Click — Simplest Steps

Project: Awareness Newsletter **Audience:** Anyone deploying the app for the first time

---

What you need before starting

Just three things:

1. **A server** — any Linux computer on the internet that you can log into. You can rent one for $5/month from DigitalOcean, AWS Lightsail, Hetzner, Linode, etc. It should be Ubuntu (easiest) or any modern Linux.
2. **A domain name** — like `awareness.mycompany.com`. Buy it from any registrar (Namecheap, GoDaddy, Cloudflare). Point its A-record at your server's IP address.
3. **An SSH key** already set up so you can log into the server (`ssh user@your-server` works without typing a password).

That's it. No GitHub, no special accounts, no extra tools.

The actual deploy — 3 steps

Step 1 — Open a terminal in the project folder

Open VS Code, then open the integrated terminal (Ctrl+`). You should be in the project folder (e.g. c:\Users\hp\OneDrive\Desktop\awback\awareness`).

Step 2 — Type one command

```
npm run deploy:server -- --host ubuntu@1.2.3.4 --domain awareness.mycompany.com
```

Replace:

- `ubuntu@1.2.3.4` with your actual server username and IP address.
- `awareness.mycompany.com` with your actual domain name.

Press **Enter**.

Step 3 — Wait about 3 minutes

The script prints progress as it goes. You'll see something like:

```
▶ Step 1/9 — Local verify gate
  ✓ npm run verify passed
▶ Step 2/9 — Build dist/
  ✓ dist/ ready
▶ Step 3/9 — SSH reachability to ubuntu@1.2.3.4
  ✓ SSH OK
▶ Step 4/9 — Detect distro and install nginx + certbot (if missing)
  ✓ Remote ready (distro: ubuntu)
▶ Step 5/9 — Copy dist/ → ubuntu@1.2.3.4:/var/www/awareness
  ✓ Web root populated at /var/www/awareness
▶ Step 6/9 — Install nginx site config
  ✓ nginx config installed and reloaded
▶ Step 7/9 — Issue Let's Encrypt certificate
  ✓ TLS issued for awareness.mycompany.com
▶ Step 8/9 — Smoke test live site
  ✓ security headers present
  ✓ health endpoint OK
▶ Step 9/9 — Write deploy log
  ✓ dist/last-deploy.json written

✓ Deployed to https://awareness.mycompany.com
```

Open the URL in a browser. The site is live.

Want to try it safely first?

Before doing the real deploy, dry-run it to see what it'll do without actually changing anything:

```
npm run deploy:server -- --host ubuntu@1.2.3.4 --domain awareness.mycompany.com --dr
```

This prints the entire plan but doesn't touch the server. Useful to confirm everything looks right.

What if you don't have a domain yet?

Skip the HTTPS step and deploy on HTTP only:

```
npm run deploy:server -- --host ubuntu@1.2.3.4 --domain 1.2.3.4 --no-tls
```

The site will be reachable at `http://1.2.3.4/`. Not secure for real use, but fine for testing.

Docker route (alternative)

If your server already has Docker installed, you have an even simpler option:

```
npm run deploy:docker -- --host ubuntu@1.2.3.4
```

The site will run on port 8080 of your server (`http://1.2.3.4:8080`).

What if it fails?

The script stops at the first failure and tells you exactly what went wrong:

- **"Cannot reach host via SSH"** → check the username + IP. Try `ssh ubuntu@1.2.3.4` manually to verify it works.
- **"npm run verify FAILED"** → some test is broken. Fix it, or pass `--skip-verify` to deploy anyway (not recommended).
- **"certbot failed"** → usually means the domain's A-record doesn't point at the server yet. Fix DNS, then re-run the same command — it picks up where it left off.

You can run the same command again as many times as you want. It's safe to re-deploy.

After deployment — what visitors do

When someone opens your live URL:

1. They see the home page immediately.
2. To use AI features, they click **Config** → enter their own AI API key → Save.
3. The key stays in their browser for that session only (security feature — never written to disk).
4. They can build newsletters, edit, send, etc.
5. When they close the tab, the API key is wiped. Next visit, they enter it again.

Cheat sheet

What you want	Command
Deploy to a fresh Linux server (most common)	`npm run deploy:server -- --host user@ip --domain you.com`
Try it without doing anything	add `--dry-run` to the above
Deploy without HTTPS (testing only)	add `--no-tls`
Deploy to a Docker host	`npm run deploy:docker -- --host user@ip`
Test the Docker image on your own laptop	`npm run deploy:docker -- --local-only`
Re-deploy after code changes	run the same command again

That's the whole flow.

Generated for the awareness-newsletter project. For the full operational guide including alternative deploy targets (Caddy, Netlify, Cloudflare Pages, Vercel), see `docs/DEPLOY.md` in the project root.